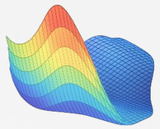


Intelligent Systems,
MIPT

Natural Language Processing

L9 / 14.04.26

- Course page
- GitHub

Content

Lecture 9: Efficient inference with Quantization, Distillation Methods, Launch

- post-training quantization methods
- quantization-aware training
- distillation methods
- inference frameworks

What is effective inference?

Efficient inference is obtaining an answer from a model with the required quality, but with minimal time and resource costs.

Overall, we want to optimize something like this:

$$\text{Utility} = \frac{\text{Quality} \times \text{Throughput}}{\text{Latency} \times \text{Cost}}$$

From an engineering perspective, there are three main bottlenecks:

- **compute:** how many matrix operations need to be performed
- **memory:** how much data needs to be read from or written to memory, and how much memory is occupied by the model and the KV cache
- **scheduling / serving:** how well we pack real user requests onto the GPU

Perfomance metrics

1. Compute metrics
   - FLOPs per request/token
   - Model FLOPs utilization
   - Tensor Core utilization
   - Prefill throughput
   - Kernel latency

$$MFU = \frac{\text{Model FLOPs per token} \times \text{Observed tokens per second}}{\text{Theoretical peak FLOPs of the hardware}}$$

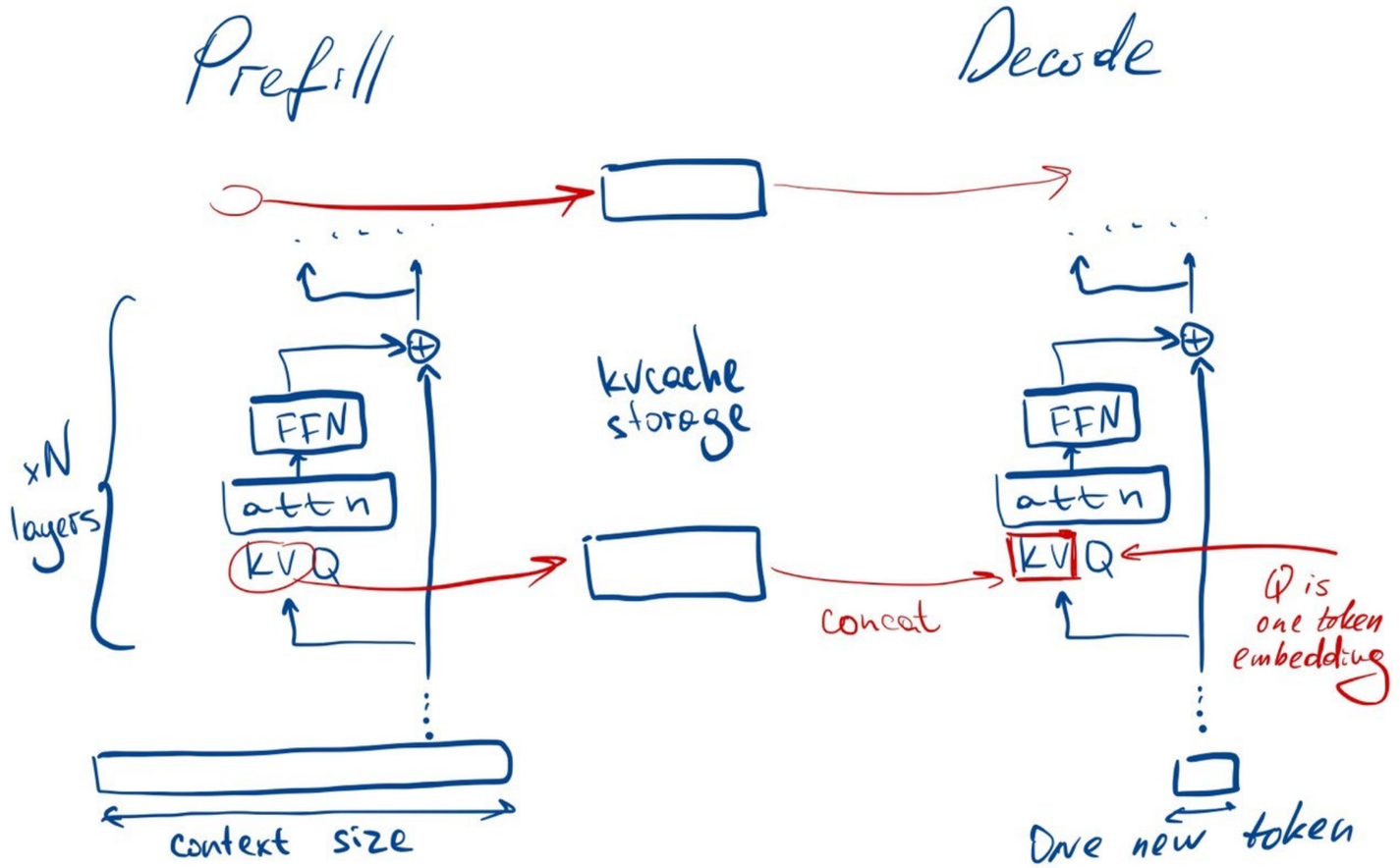
2. Memory metrics
   - VRAM usage
   - KV cache footprint
   - Memory bandwidth utilization
   - Bytes per token
   - Decode throughput

$$\text{MBU} = \frac{\text{bytes transferred per second}}{\text{theoretical maximum memory bandwidth}}$$

3. Scheduling/serving
   - Time To First Token (TTFT)
   - Time Per Output Token (TPOT)
   - Queueing delay
   - Effective batch size

$$\text{decode speed} \approx \frac{1}{\text{TPOT}}$$

Prefill and Decode



Compute

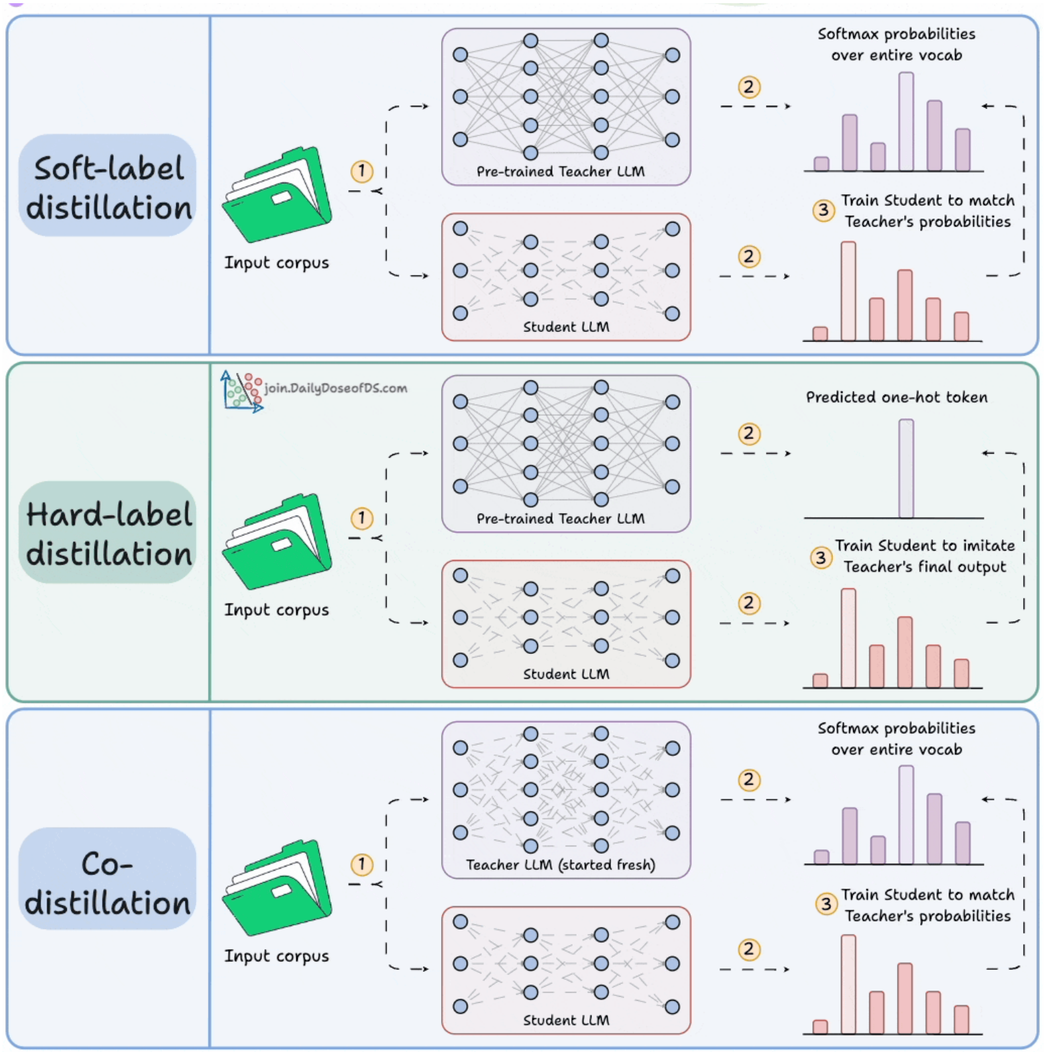
- knowledge distillation
- quantization
- speculative decoding

Knowledge distillation

- **Teacher p(y|x):** usually a LLM, e.g., LLaMA 2 (70B) offers state-of-the-art performance
- **Student q(y|x):** smaller LM, e.g., TinyLLaMA 1.1B (1.1B) struggles to match the teacher's performance through conventional training
- **Knowledge Distillation (KD):** technique to transfer the teacher's performance to the student

$$L(p(x), q_\theta(y|x)) \longrightarrow \min_\theta$$

Hard-label vs Soft-label



Formally:

$$\mathrm{KL}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i}$$

where:

- p — target distribution
- q — predicted distribution

Soft-label KD loss

$$\mathcal{L}_{\text{soft}} = T^2 \,\mathrm{KL}\left(\text{softmax}\left(\frac{z_t}{T}\right) \,\|\, \text{softmax}\left(\frac{z_s}{T}\right)\right)$$

Matches the student distribution to the teacher's full probability distribution.

Hard-label KD loss

$$\mathcal{L}_{\text{hard}} = \mathrm{CE}(\arg\max(z_t),\, z_s)$$

Matches the student to the teacher's top predicted class only.

Symmetric co-distillation

$$\mathcal{L}_{\text{co-distill}} = \sum_{i \neq j} T^2 \,\mathrm{KL}\left(\text{softmax}\left(\frac{z_j}{T}\right) \,\|\, \text{softmax}\left(\frac{z_i}{T}\right)\right)$$

Multiple models are trained jointly by matching each other's predictions.

Offline distillation

Offline distillation
Teacher model **runs in advance** on a dataset, and its outputs are **saved beforehand**.
Then the student is trained to imitate these precomputed targets.

Pipeline:
- run teacher on data
- store logits / probabilities / generated responses
- train student on saved outputs

Pros:
- simpler training pipeline
- cheaper than querying the teacher during training
- easier to debug and reproduce

Cons:
- teacher signals are frozen
- changing prompts, templates, or data often requires recomputing teacher outputs
- less adaptive to the current student behavior

Online distillation

Online distillation
Teacher model is queried **during student training**.
For each batch or rollout, the student receives fresh supervision from the teacher on the fly.

Pipeline:
- sample current batch / prompts / trajectories
- query teacher during training
- compute distillation loss immediately
- update student

Pros:
- more flexible and adaptive
- can support rollout-based and on-policy training
- teacher can guide the student on its current mistakes

Cons:
- much more expensive
- more complex infrastructure
- teacher inference can become a bottleneck

Off-policy vs on-policy

Online vs Offline
This really comes down to **when** and **how data** and **answers** are **generated**.

- Offline: Answers are generated in advance; the dataset is fixed.
- Online: Answers or the teacher signal are generated during training.

In other words, this axis represents the data collection mode.

Off-policy vs on-policy
This is another axis indicating the policy (or distribution) from which the trajectories used for the student's training are drawn. You can define the mode by asking yourself this question:

> Was the data used to calculate the loss obtained from the current student policy?

on-policy — the student learns from its own current samples
off-policy — the student learns from samples other than its own current ones

Recipe

Off-policy distillation

1. There is a prompt **x**
2. There is a certain response **y**, but it did not come from the current student
3. The teacher sends a signal based on this **y**
4. The student learns from this signal

On-policy distillation

1. We take prompt **x**
2. The student generates a response **y ~ π_S**
3. The teacher examines this specific **y**
4. A distillation signal is calculated based on it
5. The student is updated

On-policy distillation

1. Teacher scores the student's generated tokens

The student generates a response: $y = (y_1, y_2, \ldots, y_T)$
and evaluates the probability of the token actually chosen by the student: $\log p_T(y_t \mid x, y_{<t})$

2. Teacher provides top-k logits / log-probabilities

This is a richer supervision signal.
Instead of only saying whether the student token is good or bad, the teacher returns:

- the top-k most likely tokens
- their log-probabilities

3. Teacher scores the entire rollout

The teacher can assign a scalar score to the full generated response: $R_T(x, y)$

4. Teacher compares multiple student rollouts

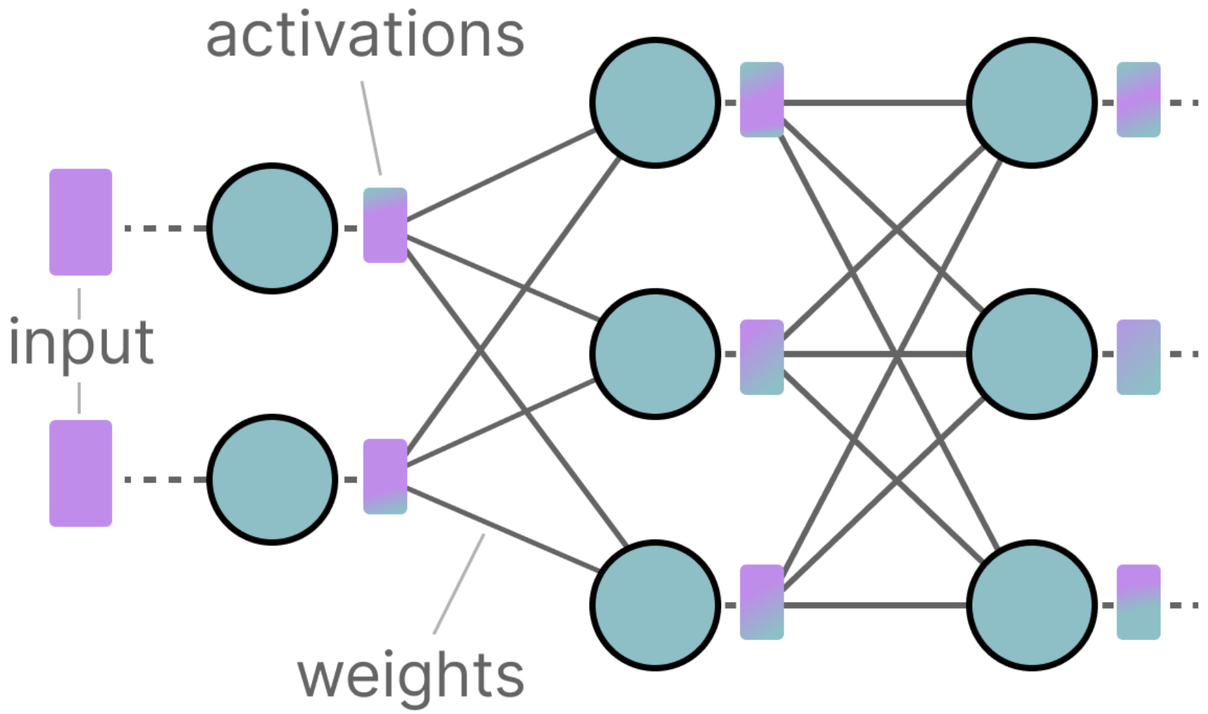
If the student samples several responses for the same prompt: $y^{(1)}, y^{(2)}, \ldots, y^{(n)}$

the teacher can: score each candidate; choose the best one; rank them

Quantization: motivation

LLMs get their name due to the number of parameters they contain. Nowadays, these models typically have billions of parameters (mostly *weights*) which can be quite expensive to store.

During inference, activations are created as a product of the input and the weights, which similarly can be quite large.



How to represent values

Float 32-bit (FP32)

$(-1)^0 \times 2^1 \times 1.5707964 = 3.1415927410125732$

higher precision

Float 16-bit (FP16)

$(-1)^0 \times 2^1 \times 1.5703125 = 3.140625$

lower precision

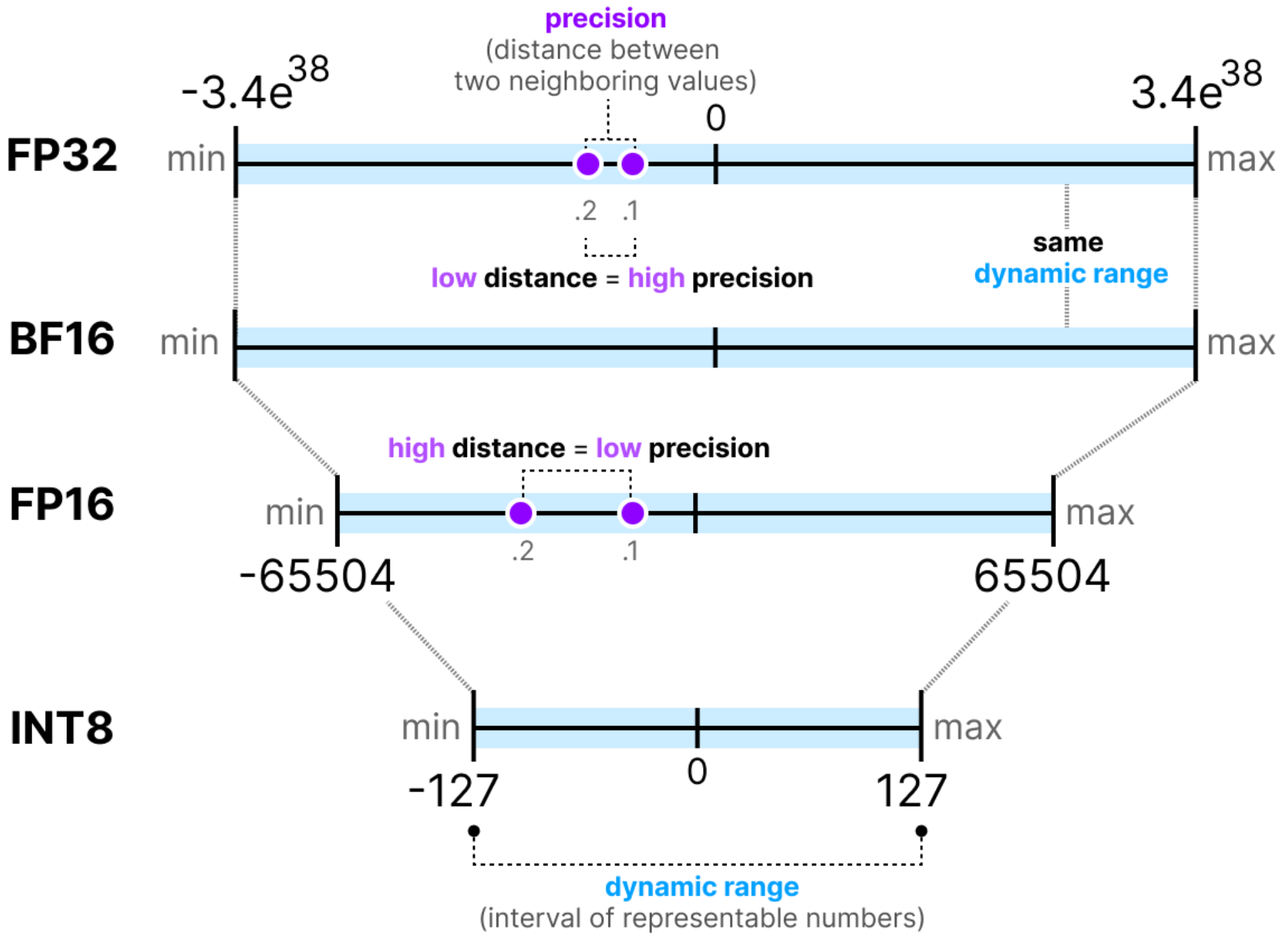
original value
3.1415927

Memory constraints

Original	75505.0	1.8e-42
64-bits	75505.0	1.8e-42
32-bits	75505.0	1.80066e-42
16-bits	inf	0.0

Numerical range

The interval of representable numbers a given representation can take is called the *dynamic range* whereas the distance between two neighboring values is called *precision*.



Memory calculation

$$\text{memory} = \frac{\text{nr\_bits}}{8} \times \text{nr\_params}$$

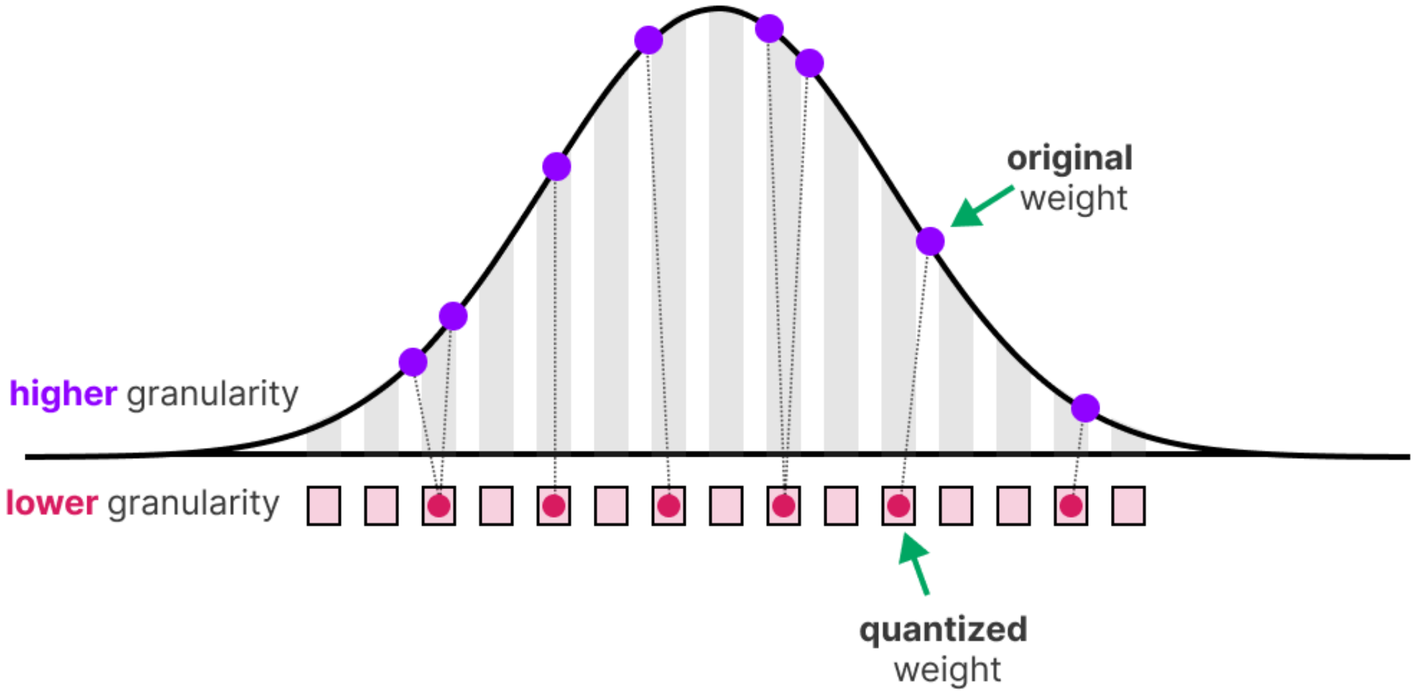
$$\textbf{64-bits} = \frac{64}{8} \times 70\text{B} \approx \textbf{560 GB}$$

$$\textbf{32-bits} = \frac{32}{8} \times 70\text{B} \approx \textbf{280 GB}$$

$$\textbf{16-bits} = \frac{16}{8} \times 70\text{B} \approx \textbf{140 GB}$$

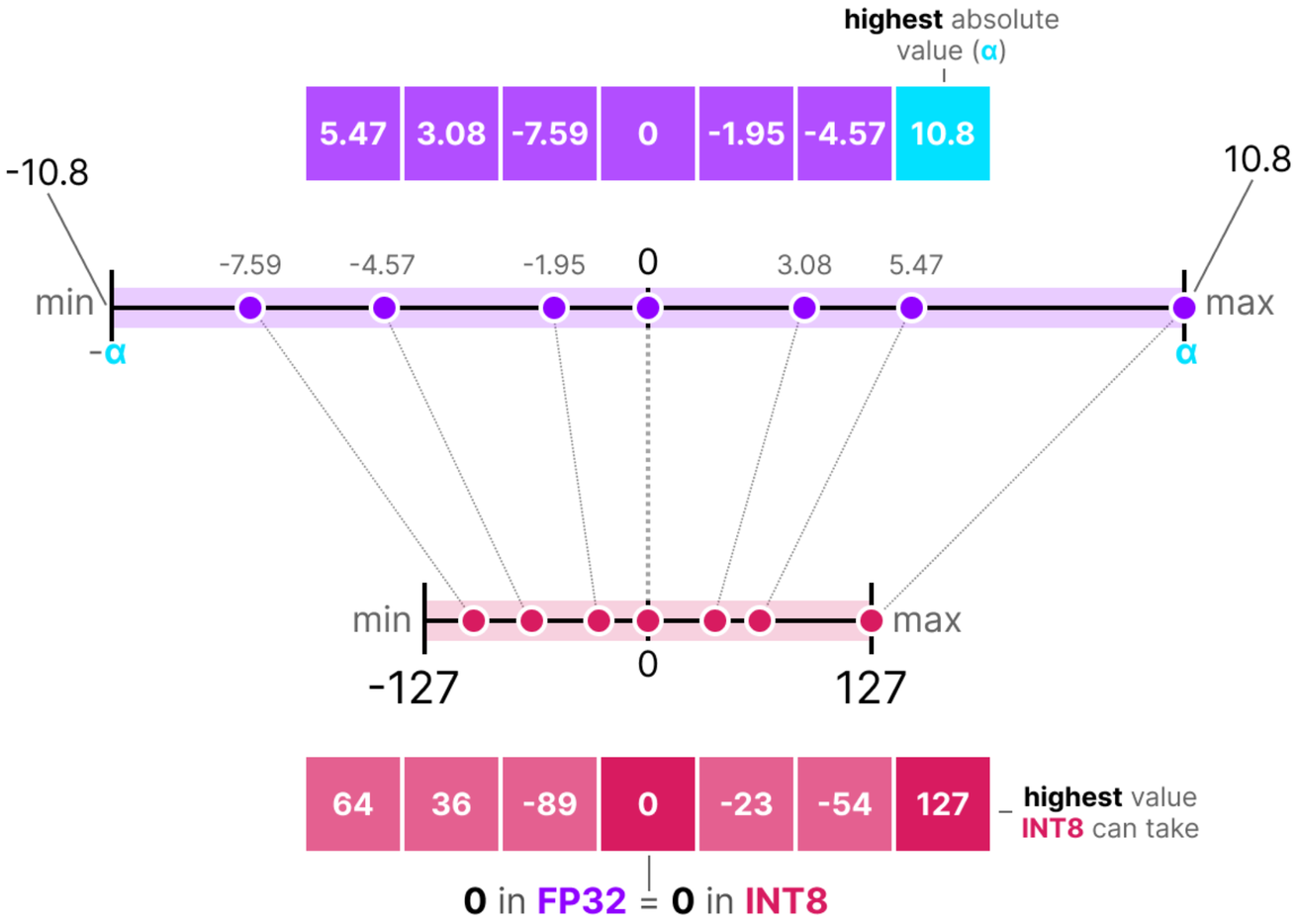
What is the quantization?

Quantization aims to reduce the precision of a model's parameter from higher bit-widths (like 32-bit floating point) to lower bit-widths (like 8-bit integers).



great overview

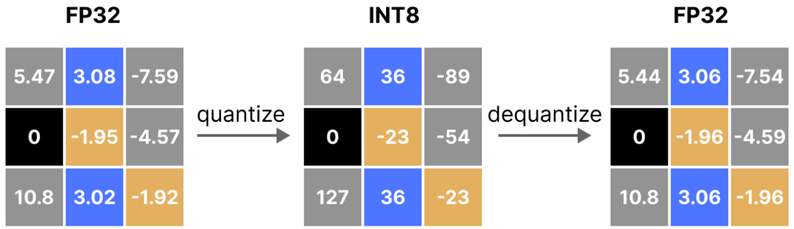
Symmetric quantization



- b is the number of **bits** that we want to quantize to (8),
- α is the highest absolute value,

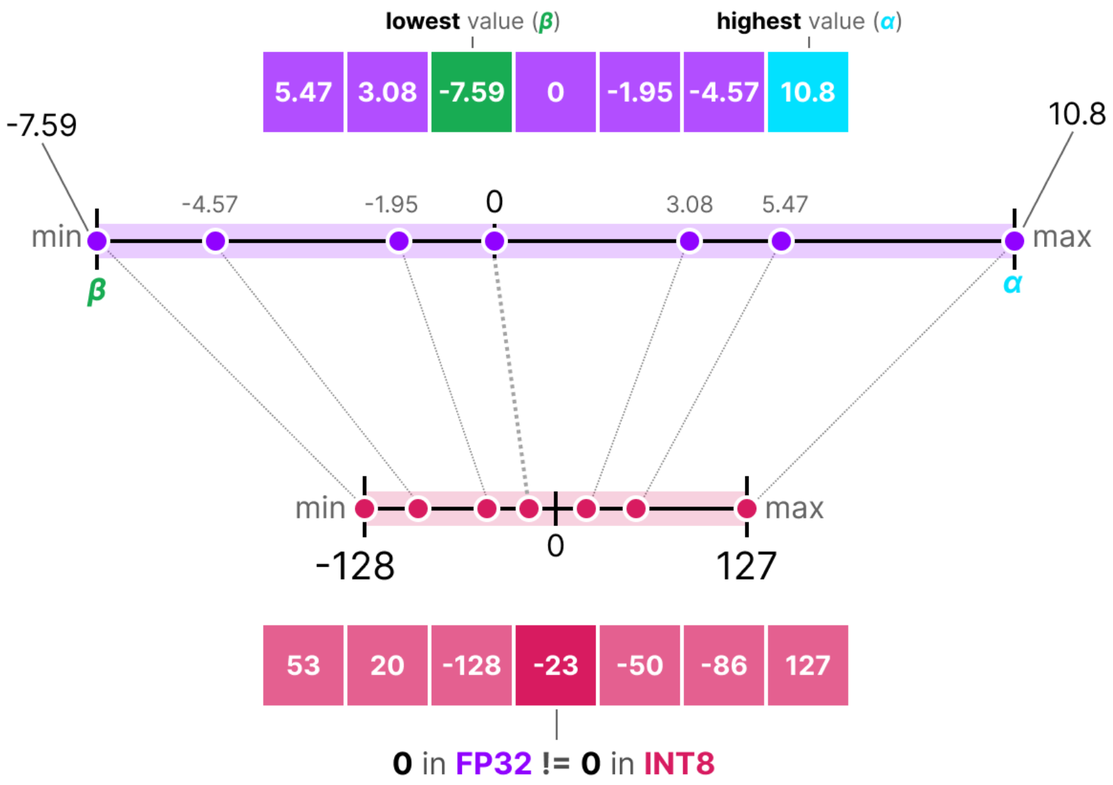
$$s = \frac{2^{b-1}-1}{\alpha} \qquad \text{(scale factor)}$$

$$x_{quantized} = \text{round}\left(s \cdot x\right) \qquad \text{(quantization)}$$



great overview

Asymmetric quantization



$$S = \frac{128 - -127}{\alpha - \beta}$$ (scale factor)

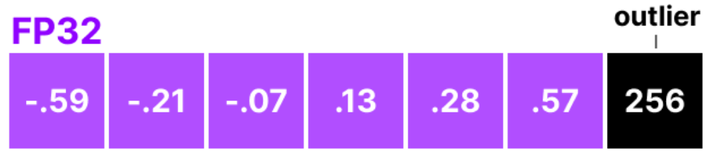
$$Z = \text{round}\left(-S \cdot \beta\right) - 2^{b-1}$$ (zeropoint)

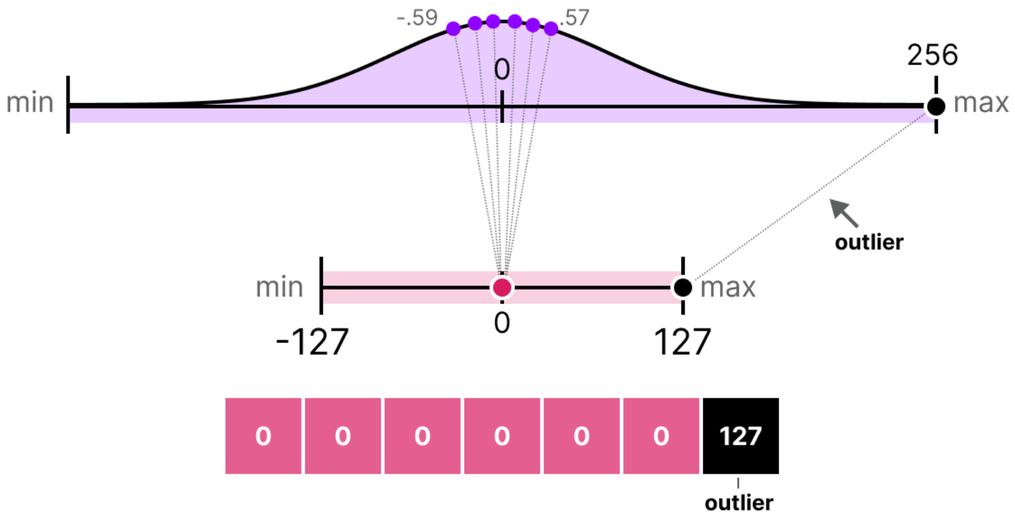
$$X_{quantized} = \text{round}\left(S \cdot X + Z\right)$$ (quantization)

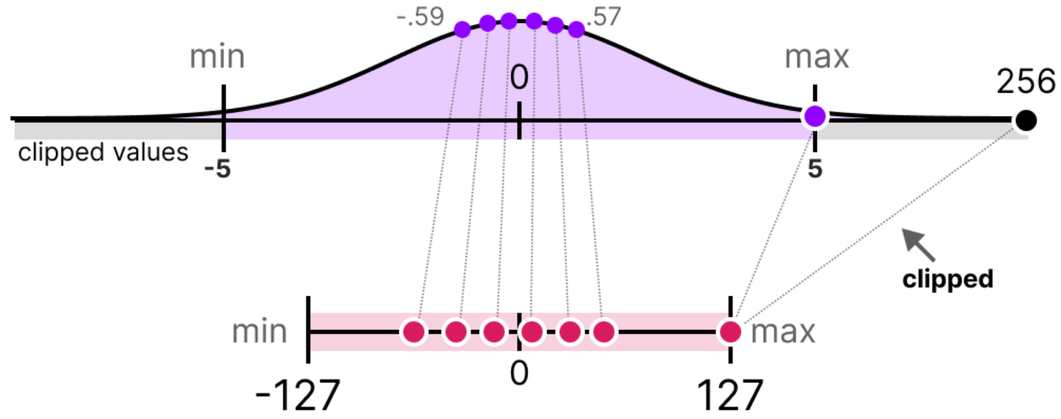
$$X_{dequantized} = \frac{\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare - Z}{S}$$ (dequantize)

great overview

Range mapping and clipping

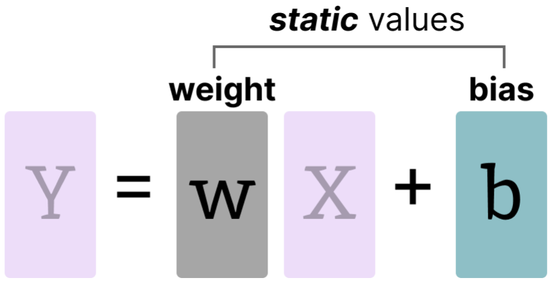






Calibration

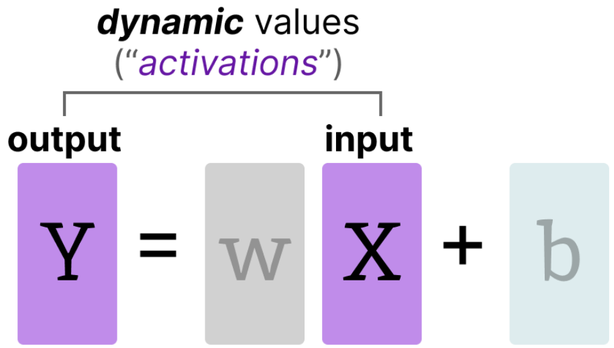
Weights and biases



For weights, which are static and known, calibration techniques for choosing the range include:

- Manually choosing a *percentile* of the input range
- Optimize the *mean squared error* (MSE) between the original and quantized weights.
- Minimizing *entropy* (KL-divergence) between the original and quantized values

Activations



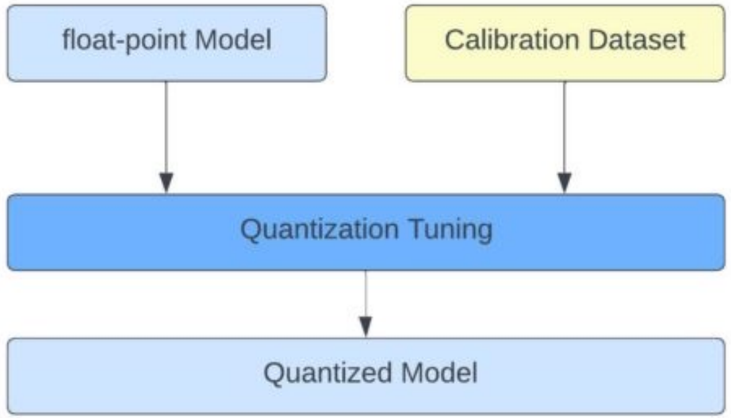
Unlike weights, activations vary with each input data fed into the model during inference, making it challenging to quantize them accurately. Broadly, there are two methods for calibrating the quantization method of the weights and activations:

- Post-Training Quantization (PTQ)
  - Quantization ***after*** training
- Quantization Aware Training (QAT)
  - Quantization ***during*** training/fine-tuning

great overview

Post-training quantization

One of the most popular quantization techniques is post-training quantization (PTQ). It involves quantizing a model's parameters (both weights and activations) **after** training the model.



Quantization of the *weights* is performed using either symmetric or asymmetric quantization.

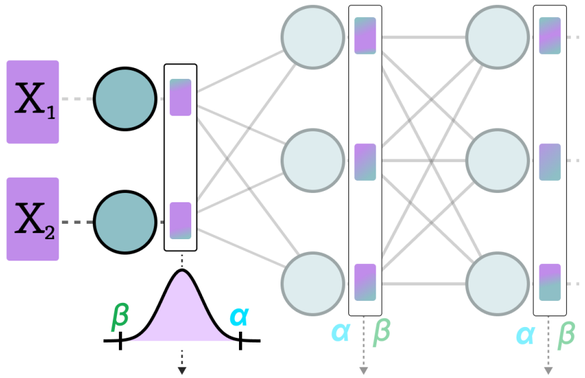
Quantization of the *activations*, however, requires inference of the model to get their potential distribution since we do not know their range.

There are two forms of quantization of the activations:

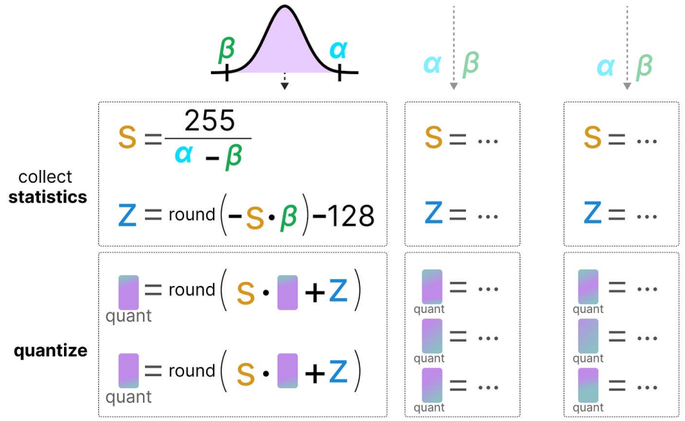
- *Dynamic* Quantization
- *Static* Quantization

Dynamic quantization

After data passes a hidden layer, its activations are collected:



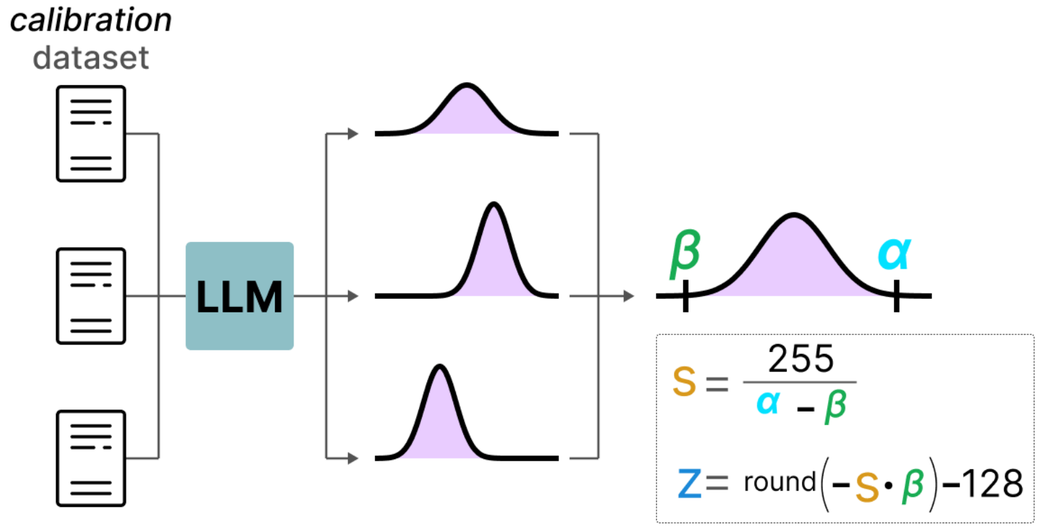
This distribution of activations is then used to calculate the *zeropoint* (***z***) and *scale factor* (**s**) values needed to quantize the output:



The process is repeated each time data passes through a new layer. Therefore, each layer has its own separate ***z*** and ***s*** values and therefore different quantization schemes.

Static quantization

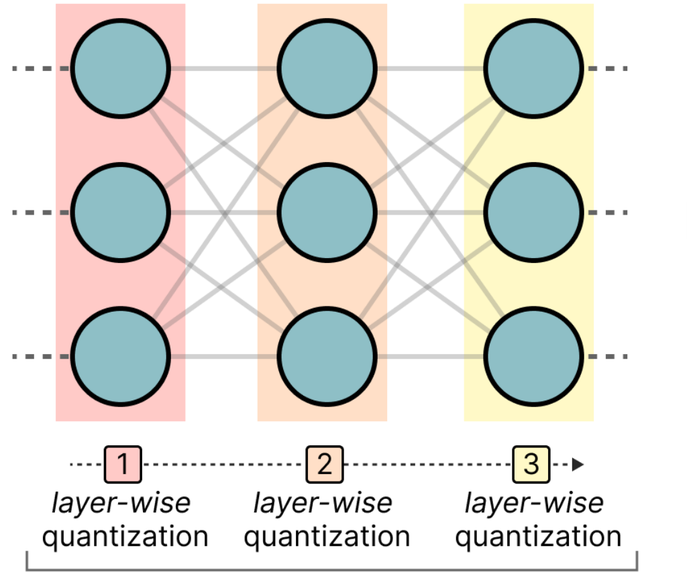
In contrast to dynamic quantization, static quantization does not calculate the *zeropoint* (***z***) and scale factor (***s***) during inference but beforehand. To find those values, a **calibration dataset** is used and given to the model to collect these potential distributions.



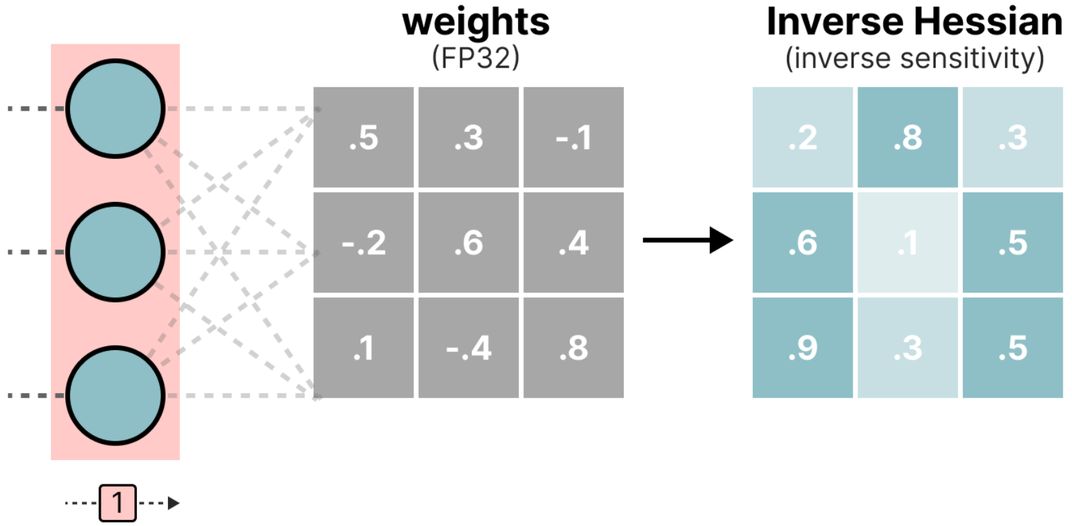
After these values have been collected, we can calculate the necessary ***s*** and ***z*** values to perform quantization during inference.

GPTQ

GPTQ is arguably one of the most well-known methods used in practice for quantization to 4-bits.



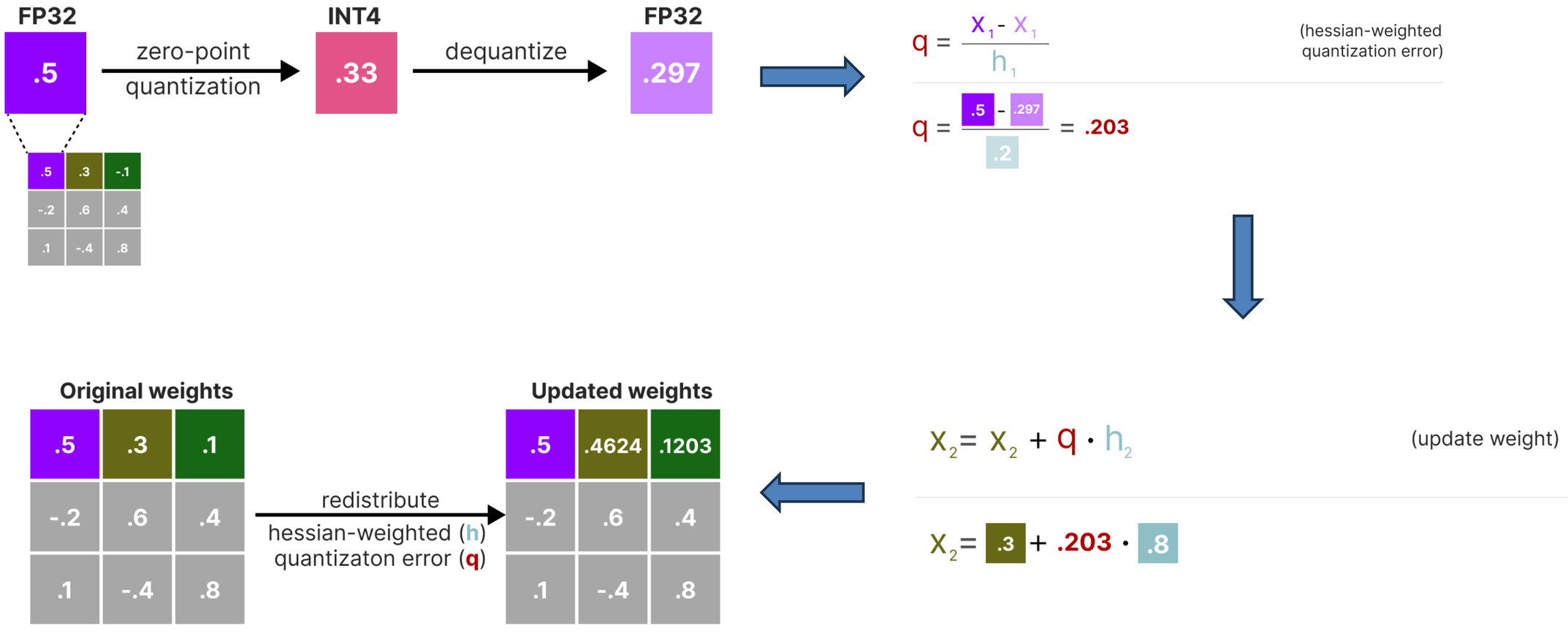
*each layer processed **independently** in **sequence***



Inverse Hessian is a second-order derivative of the model's loss function and tells us how sensitive the model's output is to changes in each weight. Simplified, it essentially demonstrates the (*inverse*) **importance of each weight** in a layer. Smaller values in the Hessian matrix are more crucial because small changes in these weights can lead to significant changes in the model's performance.

GPTQ

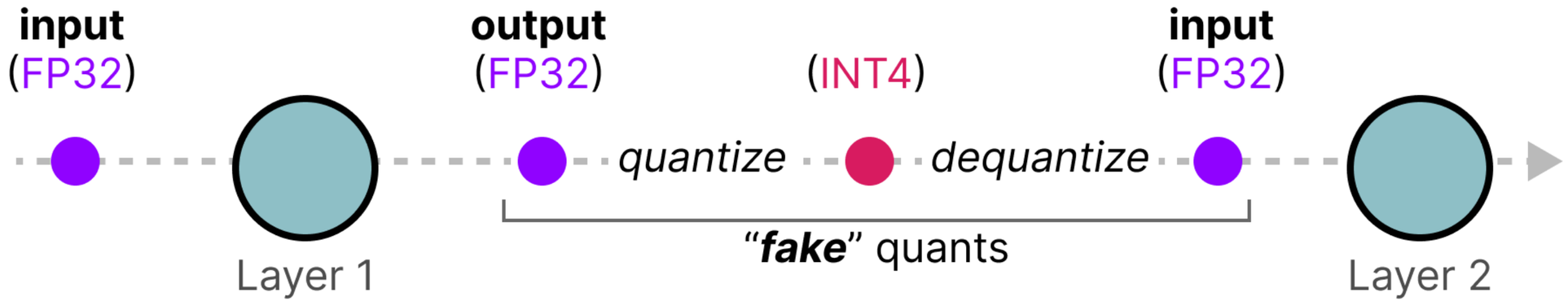
Next, we quantize and then dequantize the weight of the first row in our weight matrix:



Quantization Aware Training

Instead of quantizing a model ***after*** it was trained with post-training quantization (PTQ), QAT aims to learn the quantization procedure ***during*** training.

During training, so-called "*fake*" quants are introduced. This is the process of first quantizing the weights to, for example, INT4 and then dequantizing back to FP32:



Gradient pass

In the **forward pass**, the model applies fake quantization, for example:

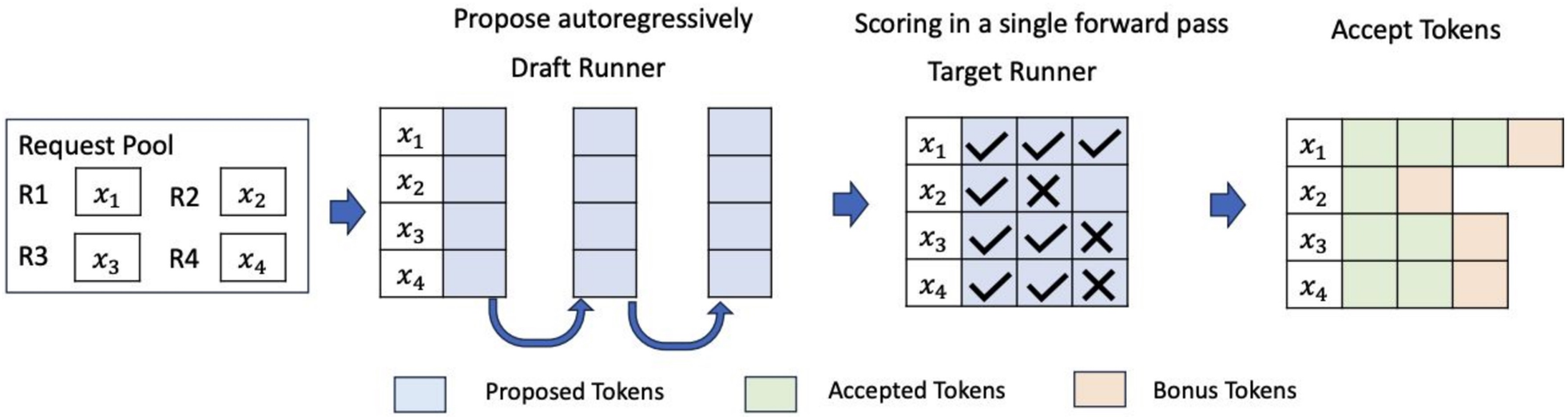
$$\hat{x} = s \cdot \text{clamp}(\text{round}(x/s), q_{\min}, q_{\max})$$

In the **backward pass**, instead of using the true derivative of round, which is zero almost everywhere, we replace it with a surrogate gradient. The most common approximation (Straight-Through Estimator) is:

$$\frac{\partial \hat{x}}{\partial x} \approx 1$$

or, more carefully, 1 inside the clipping range and 0 outside it.

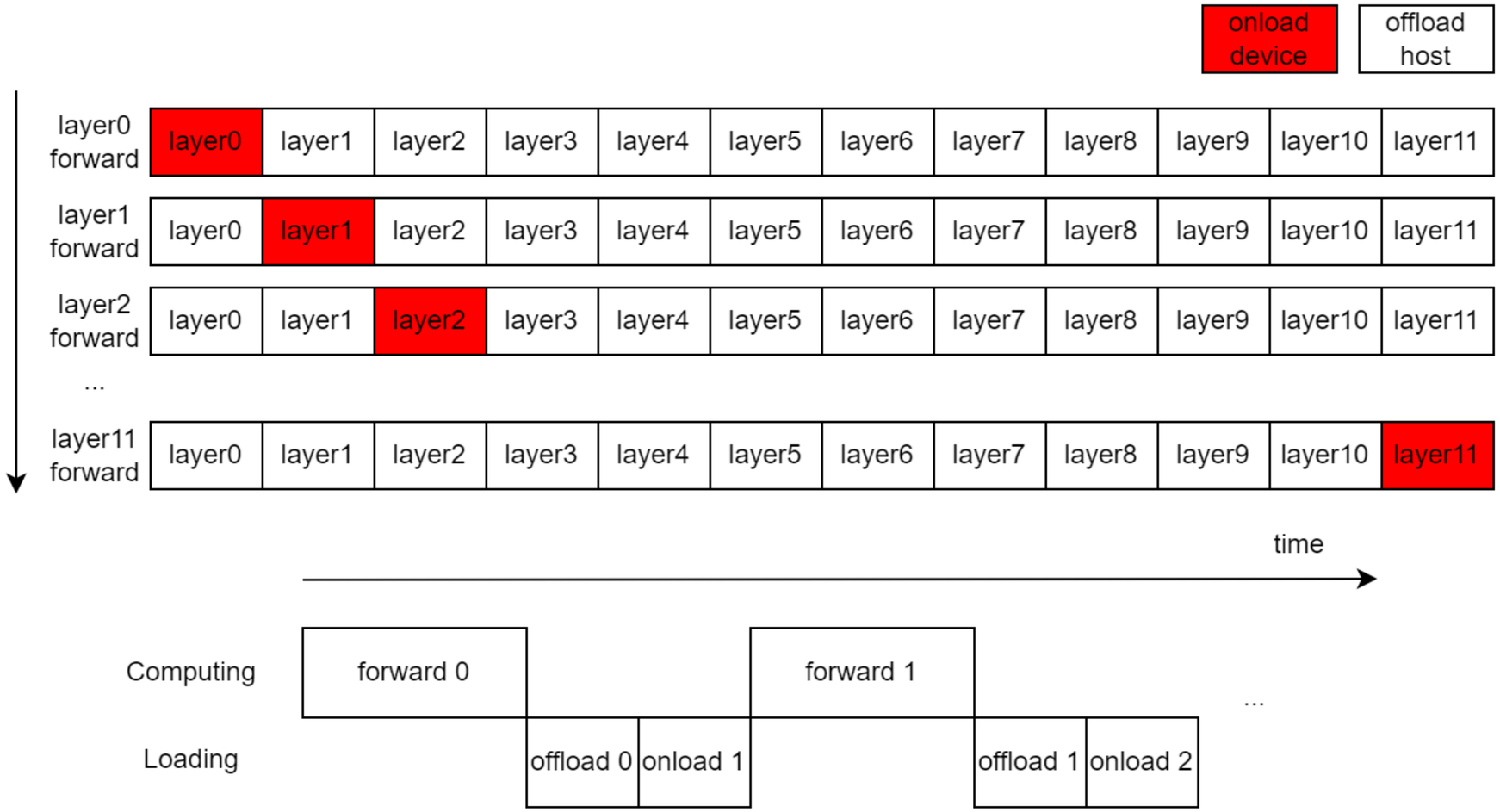
Speculative decoding



Memory

- GQA/MQA/MLA (already discussed)
- CPU offloading
- Paged attention
- Prefix caching

CPU offloading



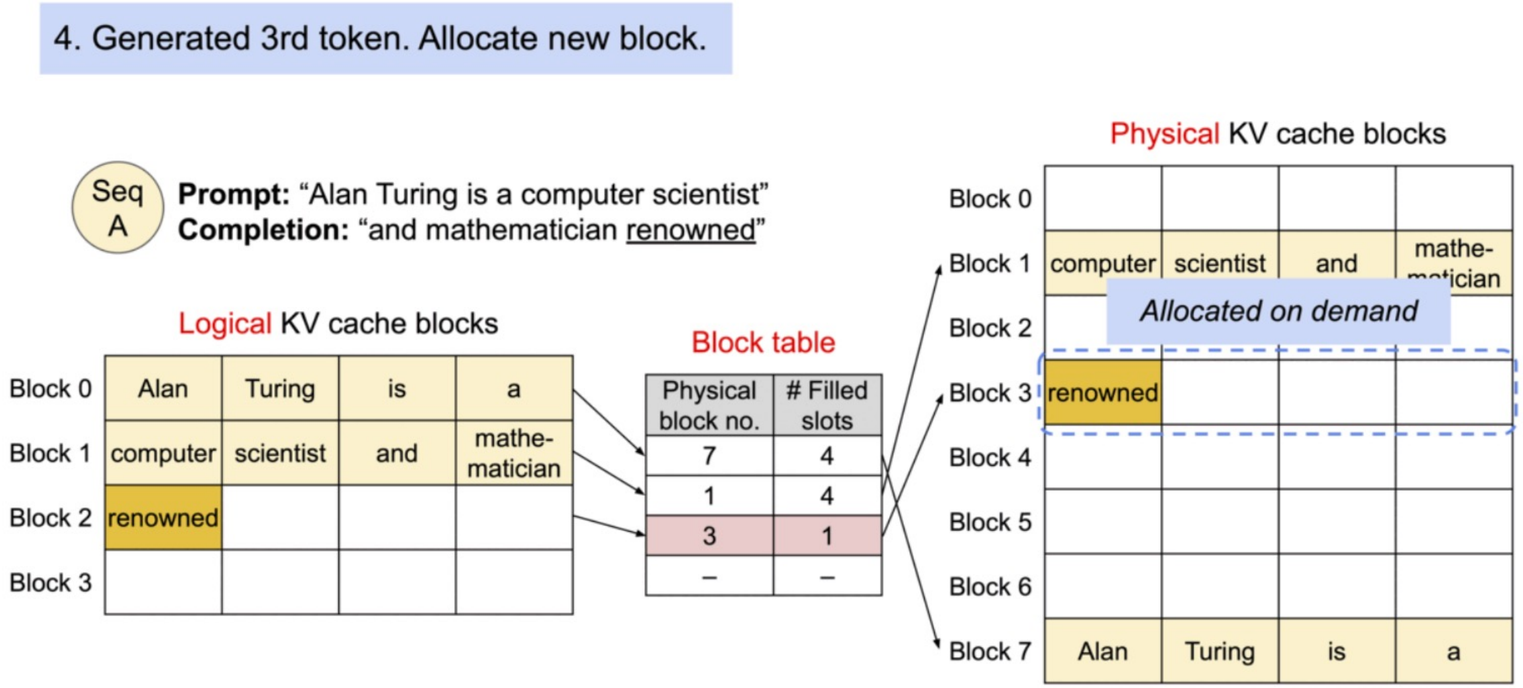
Paged Attention

Without paged attention, systems usually do something roughly like this:

- allocate one large contiguous block of memory for each request
- expand it as generation proceeds
- requests of different lengths arrive and finish at different times
- this creates holes and fragmentation in memory

As a result:

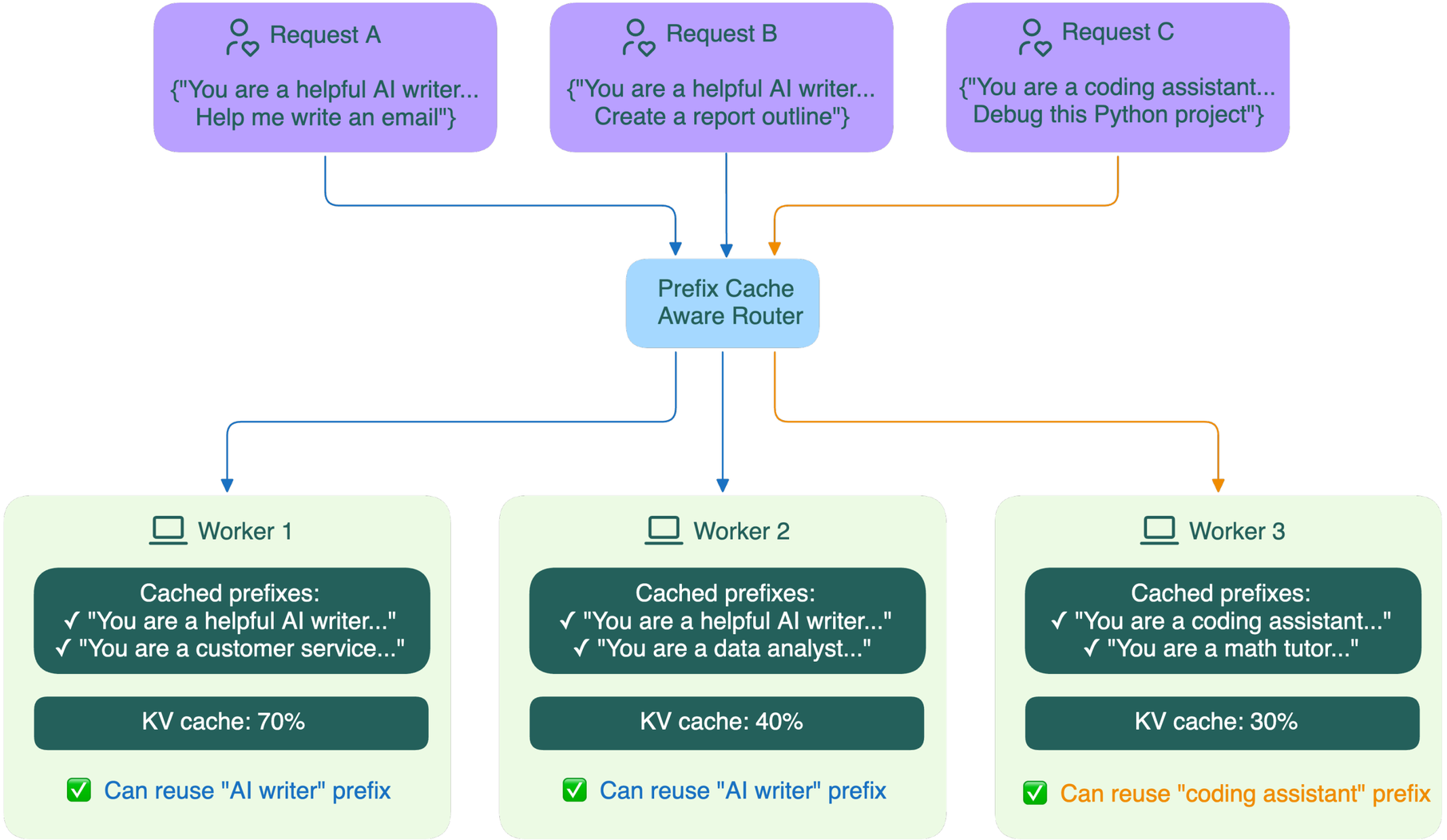
- memory is used inefficiently
- it becomes harder to serve many requests simultaneously



Example generation process for a request with PagedAttention.

blog paper

Prefix caching

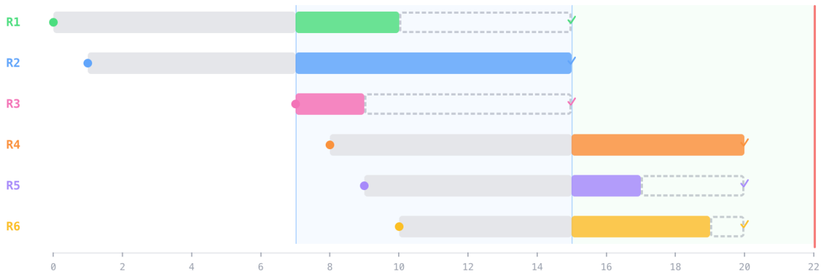


Batching

Static

batch size = 3

Collects exactly 3 requests before starting. Every request in a batch is held until the slowest one finishes.

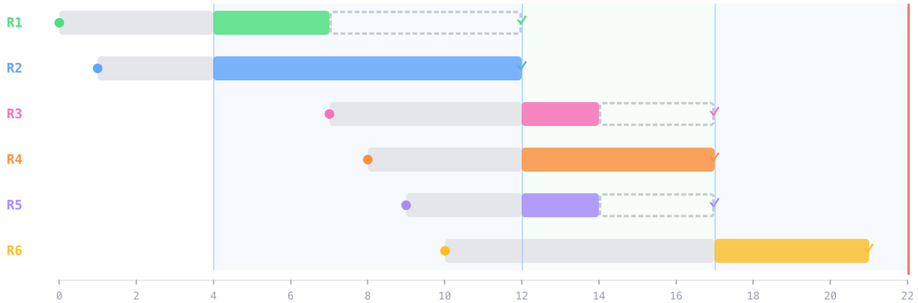


Continuous

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Slot 1	R1	R1	END	R4	R4	R4	R4	END	R7	END
Slot 2	R2	R2	R2	R2	R2	END	R5	R5	R5	END
Slot 3	R3	END	R6	R6	R6	END	R8	R8	R8	END

Dynamic

Fires when a batch reaches 3 requests OR after a 4-unit window, whichever is first.



[blog paper]

Batching

	STATIC	DYNAMIC	CONTINUOUS
Processing starts when	Batch is full	Batch full or timeout	Request arrives (if capacity is available)
Short requests held?	Yes, until slowest finishes	Yes, until slowest finishes	No, completes immediately
GPU idle between batches	Yes	Yes	No
Latency	High	Medium	Low
Throughput	Low–medium	Medium	High

blog paper

Summary

- **Efficient LLM inference is a systems problem, not just a model problem.**

- The three main bottlenecks are **compute, memory, and scheduling/serving**.

- Different methods target different bottlenecks:
  - **Compute:** quantization, pruning, distillation, speculative decoding
  - **Memory:** weight/KV-cache quantization, GQA/MQA, paged attention, CPU offloading
  - **Serving:** continuous batching, prefix caching

- The right optimization depends on the workload:
  - **interactive chat:** low TTFT and smooth decoding
  - **RAG system:** efficient KV-cache and memory usage

- In practice, the best gains often come not from a single trick, but from a **combination of model, memory, and optimizations**.

Next:

- RAG